UD 2. Introducción a JavaScript

1. Fundamentos del lenguaje JavaScript
    1.1 Entorno de ejecución de JavaScript
    1.2 Variables
    1.3 Operadores: aclaraciones
    1.4 Tipos de datos y conversiones
    1.5 strings
    1.6 Estructuras de control y bucles
    1.7 Interacción básica

2. Funciones
    2.1 Parámetros y variables locales
    2.2 Funciones como valor
    2.3 Funciones anónimas
    2.4 Funciones flecha
    2.5 Parámetros y sintaxis spread
    2.6 Ámbitos y closures

3. Objetos y prototipos
    3.1 Objetos: principios básicos
    3.2 Asignación desestructurante
    3.3 Referencia y copia
    3.4 Objetos predefinidos y métodos de primitivas
    3.5 Métodos de arrays
    3.6 this
    3.7 Prototipos y herencia
        3.7.1 Funciones constructoras
        3.7.2 Objetos enlazados
    3.8 Prototipos nativos
    3.9 Clases
    3.10 JSON

4. Módulos
    4.1 Tipos de módulos
    4.2 import y export
    4.3 Módulos en HTML
    4.4 Librerías externas: CDN
5. Programación funcional
    5.1 Fundamentos: la función filter
    5.2 find
    5.3 forEach
    5.4 map
    5.5 reduce
6. JavaScript moderno
    6.1 Modo estricto
    6.2 ¿Clases u objetos?
    6.3 Manejo de errores
    6.4 Evolución y futuro

1. Fundamentos del lenguaje JavaScript

1. Fundamentos del lenguaje JavaScript

- Lenguaje de gran complejidad. No se van a explicar todas sus características, pero haremos hincapié en las más importantes.

- La página web javascript.info es un gran libro que se mantiene al día con el estándar definido en ECMAScript (que establece el estándar de JavaScript, así como sus nuevas funciones).

- A día de hoy, ECMAScript2024 es el último estándar. Actualmente cambia cada año.

1. 1 Entorno de ejecución de JavaScript

- Puede ser usado en el navegador, y ejecutado en el cliente y en un servidor [NodeJS]. El código puede ir definido de las siguientes formas:
  - En una página HTML. Debe ir entre etiquetas **<script>** [no es necesario indicar el atributo type="script"].
  - En un archivo .js independiente. Dicho archivo puede ser ejecutado en NodeJS y cargado por una página HTML mediante **<script src="RUTA_ARCHIVO.js">.**

1. 2 Variables

- Los nombres de variables en JS deben contener únicamente letras, dígitos numéricos o los caracteres **$** y _

- El primer carácter no puede ser un dígito. Además, existe un conjunto de palabras reservadas que no pueden ser utilizadas como nombre de variables, como **function**, **let**, **for**, **if**, etc.

1. 2 Variables

- Existen tres maneras de declarar variables en JS:

1. Mediante **var**. La variable quedará definida al ámbito de la función en la que esté declarada. Si no está declarada dentro de una función, la variable quedará definida en el objeto global, del que hablaremos a lo largo de la unidad.

2. Mediante **let**. Es la forma recomendada. La variable quedará definida en el ámbito del bloque en el que esté declarada. Un bloque es un trozo de código limitado por { }. En el caso de estar definida fuera de cualquier bloque, quedará definida en el objeto global o en el módulo donde esté definida.

3. Mediante **const**. Queda definida de la misma manera que let, pero su valor no puede ser reasignado

1. 2 Variables

JS busca las variables definidas a través del llamado **ámbito léxico** o ***lexical scope***.

Cuando se hace referencia a una variable en el interior de una función [en el caso de var] o un bloque [en el caso de let], JS busca primero si existe una variable local o un parámetro con el identificador de la variable buscada:

- *si lo encuentra, lo utiliza;*
- *si no, busca en el siguiente nivel de función [o bloque], y así sucesivamente, hasta que llega a la raíz,* ***que es el objeto global*** *[programa convencional]* ***o el módulo*** *[si el código está definido en un módulo].*

1. 2 Variables

En JS moderno se tiende a usar sobre todo **let** para definir variables

Ejemplo1 – declaración de variables

- Siempre hay que minimizar el uso de variables globales.
- Las **variables globales** son accesibles, a no ser que se defina una **variable local** con el mismo nombre.
- Las **variables locales enmascaran a las globales**

Ejemplo 2 – variables locales – globales -enmascaramiento

Utiliza let SIEMPRE que vayas a usar una variable en un BUCLE, tanto si es la variable que vas a iterar como si es una variable auxiliar que vas a declarar de manera local en dicho bucle.

1. 2 Variables

Declara las variables con let, **const** o **var** antes de usarlas.

Si no, JS creará una variable automáticamente en el objeto global, con consecuencias inesperadas.

Otra solución: **usar el modo estricto**, poniendo **"use strict"** al principio del script → así se genera error si la variable no está declarada.

1. 3 Operadores: aclaraciones

El operador suma, **+**, se usa también para **concatenación** de **strings**. Si uno de los operandos es una cadena se realiza conversión:

```
1  console.log('2' + 5); // 25
2  console.log(2 + '7'); // 27
```

Para usar el operador suma hay que convertir las cadenas a números previamente.

1. 3 Operadores: aclaraciones

Esto no ocurre con el resto de operadores

```
1  console.log('2' - 5); // -3
2  console.log('4' / '2'); // 2
```

Los operadores también permiten realizar modificaciones sobre la misma variable

```
1 let a = 54;
2 a += 4; // Es lo mismo que (a = a + 4)
3 a /= 2; // Es lo mismo que (a = a / 2)
4 console.log(a); // 29
```

1. 4 Tipos de datos y conversiones

Los **tipos de datos** son:

- Tipos de datos **primitivos**:
  - **number**. Para números enteros o decimales
  - **bigint**. Para números muy grandes
  - **string**. Para cadenas de caracteres
  - **boolean**. Verdadero [true] o falso [false]
  - **null**. Valor nulo
  - **undefined**. Para variables declaradas no inicializadas [que no han recibido ningún valor]
  - **symbol**. Es un tipo de datos para crear identificadores únicos e inmutables, que se suelen utilizar para definir claves de objetos
- Tipos de datos **compuestos**:
  - **object**. Para objetos [incluidos arrays y funciones]

1. 4 Tipos de datos y conversiones

El tipado dinámico:

- Las variables pueden albergar cualquier tipo de datos.
- problemas en tiempo de ejecución. Al efectuar operaciones es posible que se produzcan conversiones de tipos:

```
let edad = prompt('¿Cuál es tu edad?'); // El usuario introduce 27. Pero "prompt" siempre devuelve un string, así que "edad" = '27'
let edadMasDiez = edad + 10; // '27' + 10 = '2710'
console.log(edadMasDiez); // 2710 en lugar de 37
// Se ha convertido el tipo de 10 a '10'. El operador '+' se ha convertido en un operador de concatenación de cadenas.
```

1. 4 Tipos de datos y conversiones

Funciones y expresiones matemáticas realizan conversiones a número [con la **excepción del operador +**, que se convierte en el operador **concatenar**]. Los casos especiales son:

Valor	Conversión a número
undefined	**NaN**
null	**0**
true/false	**1 / 0**
string	Si es una cadena vacía, **0**; si se corresponde con un nº, **el nº en cuestión**; si no corresponde a un nº válido, **NaN**

El valor **NaN** es un valor numérico que significa **Not a Number**

1. 4 Tipos de datos y conversiones

Para convertir a número podemos utilizar varios métodos

```
1  parseInt('3'); // 3
2  parseFloat('3.856'); // 3.856
3  Number('3'); // 3
4  +'3'; // 3
5
6  console.log(+'3' + 5); // 8
7
8  // Ejemplo anterior ejecutado de manera correcta
9  let edad = prompt('¿Cuál es tu edad?'); // El usuario introduce 27
10 let edadMasDiez = Number(edad) + 10; // 27 + 10 = 37. Convertimos la cadena '27' a númer
11 oonsole.log(edadMasDiez); // 37
```

1. 4 Tipos de datos y conversiones

Es posible comprobar si una cadena de texto no corresponde a un nº mediante la **función isNaN** [is not a number]

```
1 isNaN('5rt'); // true (No es un número válido)
2 isNaN('654'); // false
3 isNaN('285.32'); // false
4 isNaN('2e16'); // false
```

Conversiones a string automáticas. P.ej. La función **alert** puede tomar como parámetro un número que convierte automáticamente a string. Conversión explícita mediante función **String(valor)**.

Las **conversiones a boolean** siguen las siguientes reglas:

Valor	Conversión a boolean
0, null, undefined, NaN, “”	false
Otro valor [incluida la cadena “0”	true

1. 4 Tipos de datos y conversiones

Los operadores de **comparación de igualdad** o "**distinto de**" tienen **dos versiones**:

- **Con comprobación de tipo: ===, !==**
- **Sin comprobación de tipo: ==, !=, *conversión de tipo automática.***

```
1  let (0 == false); // true
2  let ('' == false); // true
3
4  let (0 === false); // false (distinto tipo)
5  let ('' === false); // false (distinto tipo)
```

null y **undefined** son **iguales ==** [igualdad no estricta] **entre sí y distintos a cualquier otro valor.**

1. 5 strings

Las cadenas de texto o **strings** pueden definirse mediante **comillas dobles** “ o **sencillas** ‘ .

Si se desea usar **comillas dentro de una cadena**, habrá que emplear un **entrecomillado distinto** .
Por ejemplo:

```
1   let cad = "Texto con 'comillas' dentro";
```

Existe una tercera manera de definir una cadena de texto, mediante el **acento grave [backtick]** `. Este modo tiene las siguientes **ventajas**:

- Permite crear **strings multilínea**
- Permite **incorporar expresiones** usando la estructura **${ }**.

Ejemplo 3 -- strings

1. 6 Estructuras de control y bucles

Las estructuras clásicas de programación:

- **Condicionales [if, switch]** y **bucles [for, while]** son **ya conocidas** de otros lenguajes:
- Sentencia if (javascript.info)
- Sentencia Switch
- Bucles while y for

1. 6 Estructuras de control y bucles

- **Operador ternario, ?**: Permite evaluar una condición de manera más concisa.
- Se usa para asignar un valor a una variable en función de una condición

```
let mayorDe30;
let edad = prompt('¿Cuál es tu edad?');
if (edad >= 30) {
    mayorDe30 = true;
} else {
    mayorDe30 = false;
}
```

El condicional puede reescribirse así:

```
let mayorDe30 = (edad >= 30) ? true : false;
// Se evalúa la condición (edad >= 30). Si es verdadera, se asigna a la variable "mayorDe30" el valor indicado tras '?' (true).
// Si es falsa, se asigna el valor indicado tras ':' (false).

let colores = ['azul', 'rojo', 'blanco'];
```

1. 6 Estructuras de control y bucles

Bucle for of, que se usa para **iterar sobre arrays**:

```
1  // Bucle 'for' tradiciional
2  for (let i = 0; i < colores.length; i++) {
3      console.log(colores[i]);
4  }
5
6  // Bucle 'for...of'
7  for (let color of colores) {
8      // La variable 'color' almacena el elemento del array que se está iterando
9      // Equivalencia: color = colores[i]
10     console.log(color);
11 }
```

1. 6 Estructuras de control y bucles

La estructura **for in**, aunque parecida a **for of**, se usa para **iterar sobre las propiedades de un objeto**.

Como en JS los **arrays son objetos**, también funcionará, pero **no es recomendable** usarla pues es más lenta y accede también a otras propiedades que no son los elementos del array.

1. 7 Interacción básica

alert, **prompt** y **confirm** son parte de la **API del navegador** [dentro del **objeto window**]. Por tanto, **no** están **disponibles en NodeJS**

Las tres funciones básicas de interacción con el usuario son: **alert, prompt y confirm.**

- La función **alert** muestra un mensaje en un cuadro de diálogo mediante alert("MENSAJE");
- La función **prompt** muestra un cuadro de diálogo con un texto para que el usuario introduzca un valor, y devuelve dicho valor. La función tiene dos parámetros (el segundo es opcional):

1. 7 Interacción básica

```
1  let respuesta = prompt("Texto para mostrar", "Valor por defecto del cuadro de texto (opcional)");
2  // respuesta almacena el texto escrito por el usuario en el cuadro de diálogo
3  // OJO: siempre almacenará un string. Puede que sea neceario convertirlo a otro tipo de datos.
4
5  let acepta = confirm("¿Estás seguro de que quieres continuar?");
6  // acepta almacena true si el usuario pulsa "Aceptar" y false si pulsa "Cancelar"
```

prompt devuelve un **string** si el usuario escribe algo y **null** si el usuario cancela la acción

1. 7 Interacción básica

La función **confirm** muestra un cuadro de diálogo con un texto y dos botones: **devuelve true** si el usuario pulsa en **Aceptar** y **false** si pulsa en **Cancelar**.

```
1  let acepta = confirm("¿Estás seguro de que quieres continuar?");
2  // acepta almacena true si el usuario pulsa "Aceptar"
3  // y false si pulsa "Cancelar"
```

Por último, podemos citar la función **console.log("MENSAJE")**;

Ésta escribe un mensaje en la consola. Esta función sí que está disponible tanto en el navegador como en NodeJS.

2. Funciones

2.1 Parámetros y variables locales

- Se definen mediante **function**.
- Admiten parámetros
- Permiten declarar **variables locales**
- Desde dentro de ellas se puede acceder a **variables globales** a consecuencia del **ámbito léxico**
- Los parámetros de las funciones pueden incorporar **valores por defecto**

```
1 function pintar(color="blanco") {
2     console.log(`Pintando con color ${color}`);
3 }
4
5 pintar(); // Pintando con color blanco
6 pintar("azul"); // Pintando con color azul
```

2.2 Funciones como valor

- Son objetos, y **pueden ser almacenadas en variables**

```
1 function sumar(a,b){
2     return a + b;
3 }
4
5 let suma1 = sumar(3,5); // 8
6 let funcSuma = sumar; /* funcSuma es una referencia
7                          a la función sumar, no se ejecuta*/
8 let suma2 = funcSuma(4,6); // 10
```

- Pueden ser **pasadas como parámetros** a otras funciones

2.2 Funciones como valor

- Pueden ser **pasadas como parámetros** a otras funciones

```
1  function sumar(a,b){
2      return a + b;
3  }
4  function restar(a,b){
5      return a - b;
6  }
7  function operar(operacion, a, b){
8      return operacion(a,b);
9  }
10
11 let v1 = operar(sumar, 3, 5); // 8
12 let v2 = operar(restar, 3, 5); // -2
```

- La aplicación más útil de esta característica es el uso de **callbacks**, que son funciones que se ejecutan cuando se completa una tarea.

- Ejemplo 1 – callbacks

2.3 Funciones anónimas

- No tienen nombre definido. En otros lenguajes reciben el nombre de **funciones lambda**

```
1  let sumar = function(a, b) {
2      return a + b;
3  }
4
5  let v1 = sumar(5,7); // 12
6
7  let sumar = function sumar(a, b) {
8      return a + b;
9  }
10
11 let v2 = sumar()
```

- Ambos casos muy parecidos, el primero es una función anónima y el segundo no.
- A la hora de depurar siempre viene bien que una función tenga nombre.

2.3 Funciones anónimas

- Muy común el uso de **funciones anónimas** en los **callbacks**

```
1  function confirmar(pregunta, si, no) {
2          if (confirm(pregunta))
3      si(); // Se ejecuta la función de callback pasada en el parámetro 'si'
4          else
5      no(); // Se ejecuta la función de callback pasada en el parámetro 'no'
6  }
7  // Las funciones de callback se crean in situ como anónimas
8
9  confirmar("¿Sabes qué es un callback?", function() {
10     console.log("Has respondido que sí");
11 }, function() {
12     console.log("Has respondido que no");
13 });
```

- Atención al cierre de paréntesis y llaves usando callbacks y funciones anónimas

2.4 Funciones flecha

- Las funciones flecha permiten definir funciones mediante la siguiente sintaxis

```
let func = (par1, par2, ... parN) => expresión;
```

- Así, las siguientes funciones:

```
function sumar(a, b) {
    return a + b;
}

function restar(a, b) {
    console.log("Restando...");
    return a - b;
}
```

- se podrían expresar así:

```
// El cuerpo de la función solo tiene una línea.
// Se pueden omitir las llaves y el 'return'
let sumar = (a, b) => a + b;

// El cuerpo de la función tiene varias líneas.
// Se deben incluir las llaves y el 'return'
let restar = (a, b) => {
    console.log("Restando...");
    return a - b;
}
```

2.4 Funciones flecha

No son únicamente versiones simplificadas de las funciones convencionales. Tienen unas características especiales. Algunas se comprenderán más adelante:

- Son siempre funciones **anónimas**.
- No admiten los métodos **call**, **apply** y **bind**.
- No tienen su propio **this**. No adecuadas para métodos de objetos.
- No pueden usarse como constructores con **new**.

2.5 Parámetros rest y sintaxis spread

- En ocasiones, es interesante pasar a una función **un número variable de argumentos**. Ejemplo función que calcule una media aritmética. En esos casos es útil el uso del operador …
- Ejemplo 2 – parámetros rest

```
1  function sumar(a, b) {
2      return a + b;
3  }
4
5  let numeros1 = [4, 6];
6  sumar(...numeros1); // 10
7
```

Si la función tiene varios parámetros, únicamente **el último** puede usarse como **parámetro REST**

Operación inversa. Convertimos un array a un conjunto de parámetros.

2.5 Parámetros rest y sintaxis spread

- **Copia de arrays** o **creación de arrays nuevos** a partir de existentes y elementos nuevos, gracias al operador …

```
let colores1 = ['azul', 'rojo', 'verde'];
let colores2 = colores1; // no es una copia, es una referencia
// solo hay un array y dos variables que apuntan a él
let colores3 = [...colores1]; // copia de array en uno nuevo
// hay dos arrays diferentes
let colores4 = ['cian', ...colores1, 'amarillo'];
// colores4 almacena un array nuevo con nuevos elementos
// Modifico el primer array
colores2[0] = 'blanco';
console.log(colores1[0]); // blanco
console.log(colores2[0]); // blanco (mismo array)
console.log(colores4.length); // 5
console.log(colores4); // ['cian', 'azul', 'rojo', 'verde', 'amarillo']
```

2.5 Ámbitos y closures

- Funciones que pueden acceder y usar valores declarados fuera de su contexto / ámbito: **un closure**.

Imagina que estás jugando al escondite con tus amigos.

1. Caja Grande:
Piensa en una casita de juguete donde decides esconder un dulce. Esa casita tiene varias habitaciones.

2. Caja Pequeña:
En una de esas habitaciones, hay una cajita donde pones el dulce. Sólo tú sabes dónde está esa cajita.

3. La Llave:
La habitación y la cajita son como una clave secreta. Aunque te salgas de la casita de juguete, sigues recordando dónde escondiste el dulce.

Ahora, en JavaScript:
•La **casita** es una función grande.
•La **cajita** es otra función dentro de la casita.
•El **dulce escondido** es la información especial (una variable) que la cajita puede recordar, incluso si sales de la casita.

2.5 Ámbitos y closures

- Funciones que pueden acceder y usar valores declarados fuera de su contexto: un **closure**.
- Valores definidos fuera de su cuerpo
- Las funciones son ciudadanos de primera clase en el lenguaje y pueden ser utilizadas como cualquier otro objeto: anidadas, pasadas como argumento, como valor de retorno, etc.

```
1  function miFuncion() {
2      let count = 1
3      function contador() {
4          count++
5          console.log(count)
6      }
7      setInterval(contador, 2000)
8  }
9  miFuncion()
```

2.5 Ámbitos y closures

```
function creaContador() {
    let cuenta = 0; // Aquí escondemos el dulce

    return function() {
      cuenta += 1; // Esta función pequeña recuerda dónde está el dulce
      console.log(cuenta);
    }
  }

   let contador = creaContador(); // Entramos a la casita y encontramos la cajita

   contador(); // 1 (recuerda dónde está el dulce)
   contador(); // 2 (sigue contando desde el dulce que escondimos)
```

La función creaContador es la casita grande, y la función devuelta es la cajita pequeña que recuerda la cuenta. Aunque uses contador() fuera de creaContador, la función pequeña aún recuerda la variable cuenta y sigue contando desde allí.

¡Eso es una **closure**, o **clausura**!

2.5 Ámbitos y closures

```
1  function crearContador() {
2      let cuenta = 0;
3      // Función closure
4      return function() {
5          // Referencia a la variable "cuenta" del ámbito exterior a la función
6          return cuenta++;
7      };
8  }
9
10 let contador1 = crearContador();
11 console.log(contador1()); // 0
12 console.log(contador1()); // 1
13 console.log(contador1()); // 2
14
15 let contador2 = crearContador();
16 console.log(contador2()); // 0
17 console.log(contador2()); // 1
18
```

crearContador devuelve una función al ejecutarse.

contador1 y **contador2** hacen referencia a funciones que son closures, pues guardan una referencia a la variable cuenta.

3. Objetos y prototipos

3.1 Objetos: principios básicos

- Los **objetos** son el tipo de datos complejo disponible en JS. Un objeto está formado por colecciones de datos definidas como propiedades.
- Declaración de un objeto mediante un **literal**:

```
let coche = {
    color: "rojo",
    marca: "seat",
    arrancar: function(){
        console.log("Arrancando");
    }
}
```

- También podemos declarar un objeto llamando a su constructor por defecto:

```
let miObj = new Object();
```

- Al acceder a una propiedad inexistente de un objeto, no se mostrará error pero sí se devolverá **undefined**.
- Con el operador **in** podemos ver si una propiedad está definida en un objeto o no:

```
console.log("color" in coche);
```

- Devolverá true al encontrarse definida.

3.2 Asignación desestructurante

- En ocasiones nos puede interesar extraer determinadas propiedades de un objeto o elementos de un array y asignarlas a variables independientes.

En Arrays

```
let array1 = ["valorA", "valorB", "valorC"];

// En arrays las variables se asocian por posición
let [a, b, c] = array1;

console.log(a); // valorA
console.log(b); // valorB
console.log(c); // valorC

// Ignorar un elemento
let [d, , e] = array1;
console.log(d); // valorA
console.log(e); // valorC

// Utilizar el operador rest: 'g' almacenará un array
let [f, ...g] = array1;
console.log(f); // valorA
console.log(g); // ["valorB", "valorC"]
```

En Objetos

```
let objeto1 = {
    pA: "valorA",
    pB: "valorB",
    pC: "valorC"
};

// En objetos las variables es asocian por nombre
let {pA, pB, pC} = objeto1;

console.log(pA); // valorA
console.log(pB); // valorB
console.log(pC); // valorC

let resto;
// OJO: en este punto hemos definido 'resto' en la línea anterior
// No podemos poner 'let' en la línea siguiente porque 'pA' ya ha sido definida al principio
// Por tanto, debemos poner la expresión entre paréntesis
({pA, ...resto} = objeto1); // Utilizar el operador rest: 'resto' almacenará un objeto
console.log(pA); // valorA
console.log(resto); // {pB: "valorB", pC: "valorC"}
```

Ejemplo 1 – asignación desestructurante

Ejemplo 2 – asignación desestructurante

3.3 Referencia y copia

- Las variables **no almacenan objetos**, sino que guardan referencias a ellos.

```
let objeto1 = {nombre: "David"}; // Se crea un objeto. 'objeto1' apunta a él
let objeto2 = objeto1; // NO SE CREA NINGÚN OBJETO NUEVO. 'objeto2' apunta al mismo objeto que 'objeto1'

let objeto3 = {}; // Se crea un objeto vacío
let objeto4 = {}; // Se crea otro objeto vacío
Object.assign(objeto3, objeto1); // Copia de 'objeto1' a 'objeto3'
Object.assign(objeto4, objeto1, {apellido: "Pérez"}); // Copia de 'objeto1' a 'objeto4' junto con una propiedad adicional

objeto1.nombre = "Juan"; // Cambio en 'objeto1'
objeto4.nombre = "Pablo"; // Cambio en 'objeto4'

console.log( objeto2.nombre ); // Juan ('objeto2' apunta al mismo objeto que 'objeto1')
console.log( objeto3.nombre ); // David ('objeto3' apunta a un objeto distinto)

objeto1 = null; // 'objeto1' ya no apunta al objeto. El objeto sigue existiendo.
objeto2 = null; // 'objeto2' ya no apunta al objeto. El primer objeto se elimina porque ya no está referenciado
```

- Para copiar un objeto, se puede usar **Object.assign(OBJETO_DESTINO, OBJETO_ORIGEN)**

  Ejemplo 3 – referencia y copia

3.4 Objetos predefinidos y métodos de primitivas

- Los tipos de datos primitivos pueden convertirse a objetos. JS define un mecanismo de herencia basada en prototipos, de tal forma que los objetos pueden acceder a propiedades definidas en sus prototipos.
- No son lo mismo los **tipos de datos primitivos string y number** que los objetos predefinidos **String** y **Number** que se aplican a estos tipos de datos primitivos para así poder heredar sus métodos.

```
1  //Gracias a los objetos predefinidos podemos ejecutar el siguiente código:
2  let cadena = "miCadena";
3  console.log(cadena.length);
4  console.log(cadena.toUpperCase());
5  //se aplican métodos del objeto genérico String al tipo primitivo string
```

3.5 Métodos de arrays

- Los **arrays** en JS son **objetos** y tienen **acceso por herencia** a una serie de métodos muy útiles.

```
//Gracias a los objetos predefinidos podemos ejecutar el siguiente código:
let cadena = "miCadena";
console.log(cadena.length);
console.log(cadena.toUpperCase());
//se aplican métodos del objeto genérico String al tipo primitivo string
```

```
let array3 = [1, 2, 3, 4, 5];
let array4 = array3.slice(1, 4); // Extrae los elementos entre las posiciones 1 y 4 (sin incluir el 4)
// array3 no se modifica
// array4 = [2, 3, 4]

let array5 = [1, 2];
let array6 = [8, 9];
let array7 = array5.concat(array6); // [1, 2, 8, 9]
//unión de arrays con concat
// array5 y array6 no se modifican

//indexOf devuelve el índice del array donde se encuentre
//el elemento pasado como argumento
array7.indexOf(8); // 2
array7.includes(1); // true
array7.includes(3); // false
```

Ejemplo 4 – métodos de arrays

3.6 this

this es una palabra reservada **usada dentro de los métodos de los objetos para hacer referencia al objeto sobre el que se llama a dicho método en el lugar de ejecución, no en la definición**. Su comportamiento es distinto al de otros lenguajes

Ejemplo 5 – this

```
// Esta función tiene dos parámetros y además hace referencia a 'this'
function saludar(mensajeInicio, mensajeFinal) {
    return mensajeInicio + this.nombre + mensajeFinal;
}

// Objeto sobre el que realizar pruebas
let usuario1 = {nombre: "David"};

// Ejecución de la función sin pasar referencia a ningún objeto.
saludar("Hola ", ", ¿qué tal estás?"); // Hola undefined, ¿qué tal estás?
/*MÉTODOS CALL APLY Y BIND para especificar*/
// Ejecución de la función mediante 'call': los parámetros se pasan como lista
saludar.call(usuario1, "Hola ", ", ¿qué tal estás?"); // Hola David, ¿qué tal estás?

// Ejecución de la función mediante 'apply': los parámetros se pasan en un array
saludar.apply(usuario1, ["Hola ", ", ¿qué tal estás?"]); // Hola David, ¿qué tal estás?

// Enlace explícito de la función al objeto 'usuario1'
//bind no invocará la función directamente, sino que cambiará el valor this
//dentro de la función y devolverá la instancia de la función modificada.
let saludaADavid = saludar.bind(usuario1);
// La ejecución de la función se realiza sin pasar referencia. Sin embargo,
// ésta ya estaba pasada mediante 'bind'.
saludaADavid("Hola ", ", ¿qué tal estás?");  // Hola David, ¿qué tal estás?
```

3.6 this

- **this** puede a veces no funcionar como se espera, sobre todo cuando se usan **callbacks**. En el siguiente ejemplo usamos la función **setTimeout**, que ejecuta una función pasados ciertos milisegundos.

- **setTimeout** es una función asíncrona (espera y luego ejecuta la función pasada como primer parámetro). La llamada a la función se hace sin referencia a ningún objeto, así que **this** no apunta al objeto que esperábamos y sí al **objeto global** (**window** en el navegador, **global** en **Node**, **globalThis** en cualquier otro entorno)

3.6 this

```
let usuario = {
    nombre: "Miguel",
    saluda: () => {  //Las funciones flecha no definen su propio this
        return this.nombre; //este método no tiene this por tanto
    }
}

let a = usuario.saluda();
console.log(a);

let ciudadano = {
    nombre: "David",
    saludaConRetardo: function(){
                            setTimeout(function(){
                                        console.log(this.nombre); //intenta mostrar nombre de usuario
                                            //pero setTimeout llama a la función sin referencia a ningún objeto
                                        }, 1000); //Retardo de 1000ms (1s)
                                    }
}

ciudadano.saludaConRetardo(); //undefined
```

3.6 this

Para conseguir que **this** haga referencia al objeto deseado, se puede usar un **enlace explícito con bind**

```
let ciudadano = {
    nombre: "David",
    saludaConRetardo: function(){ //el método 'saludaConRetardo' sí tiene como contexto 'usuario' cuando sea ejecutada
                            setTimeout(function(){
                                        console.log(this.nombre);
                                        }.bind(this), 1000);
                                    }//al usar 'bind', 'setTimeOut' tendrá el mismo contexto
}

ciudadano.saludaConRetardo(); //David
```

3.6 this

Una solución alternativa es usar una **función flecha** que no tiene **this**.

```
let ciudadano = {
    nombre: "David",
    saludaConRetardo: function(){ //el método 'saludaConRetardo' sí tiene como contexto 'usuario' cuando sea ejecutada
                            setTimeout(() => {
                                        console.log(this.nombre);
                                        }, 1000);
                                    }//al usar una función flecha que no tiene this, JS lo busca en el ámbito exterior
}                                    //es decir en el cuerpo de la función 'saludaConRetardo'

ciudadano.saludaConRetardo(); //David
```

3.6 this

Reglas para determinar **this**:

1. Si se usa el operador **new**, es el objeto devuelto por el constructor
2. Si se usa un **enlace explícito** (**call**, **apply** o **bind**), es el objeto especificado
3. Si se usa un **enlace implícito**, es el objeto especificado
4. Si no se aplica nada de lo anterior, es el **objeto global**, o **undefined** en **modo estricto**.

3.7 Prototipos y herencia

- JS es un **lenguaje orientado a objetos** que usa un sistema de **herencia basada en prototipos**.
- **No hay clases**, hay **objetos enlazados a otros objetos**, llamados **prototipos**. Las propiedades de los prototipos no se heredan (no se copian a los nuevos objetos).
- Los objetos acceden a ellas mediante un mecanismo llamado **delegación**.
- **Delegación**: Si se intenta acceder a una propiedad (incluyendo funciones) de un objeto, JS busca primero si está definida en el objeto, si no lo está busca en su prototipo.

```
let usuario = {
    nombre: "Miguel",
    saluda: () => {  //Las funciones flecha no definen su propio this
        return this.nombre; //se coge el this del ámbito exterior (objeto usuario)
    }
}

//El método 'toString' funciona, aunque no se haya definido
//Está definido en 'Object.prototype'
console.log(usuario.toString());
```

3.7 Prototipos y herencia. Funciones constructoras

- Con las funciones constructoras tenemos una técnica para crear objetos basados en prototipos gracias a la propiedad **prototype**.
- Una función constructora es una función que define una serie de propiedades y métodos. Para hacer referencia a las propiedades y métodos se usa **this**.
- Una vez definida para crear un objeto con función constructora se usa el operador **new**.
- Normalmente, los constructores no tienen una sentencia return. Su tarea es escribir todo lo necesario al this, y automáticamente este se convierte en el resultado.

3.7 Prototipos y herencia. Funciones constructoras

```
1  function Usuario (nombreP, edadP) //nombre de función constructora en mayúsculas
2  {
3      this.nombre = nombreP;
4
5      if (edadP >=0)
6          this.edad = edadP;
7      else
8          this.edad = 0;
9
10     this.muestraUsuario = function () {
11         console.log(`Nombre: ${this.nombre}, Edad: ${this.edad}`);
12     }
13 }
14
15 let user1 = new Usuario ("Fulano", -3);
16
17 user1.muestraUsuario(); //Nombre: Fulano, Edad: 0
```

Ejemplo 6 – funciones constructoras de objetos

3.7 Prototipos y herencia. Funciones constructoras

- Si hay una sentencia **return**, entonces la regla es simple:
- Si **return** es llamado con un objeto, entonces se devuelve tal objeto en vez de **this**.
- Si **return** es llamado con un **tipo de dato primitivo**, es ignorado.
- En otras palabras, **return con un objeto devuelve ese objeto**, en todos los demás casos se devuelve **this**.
- La función constructora define las propiedades que tendrán los objetos creados a partir de ella.

3.7 Prototipos y herencia. Funciones constructoras

Son una técnica para crear objetos basados en prototipos pues la función constructora *creará un enlace de dichos objetos a* ***NOMBRE_FUNCION.prototype****, que será el prototipo al que están enlazadas. Si se crea una* ***nueva propiedad*** *(incluyendo funciones)* ***en el prototipo****,* ***todos los objetos podrán acceder a ella****. Si se* ***modifica*** *alguna* ***propiedad definida en el prototipo, se copia al objeto*** *y no se modifica la del prototipo.*

3.7 Prototipos y herencia. Objetos enlazados

- La segunda técnica para crear objetos basados en prototipos se basa en la función **Object.create** y utiliza exclusivamente objetos, no funciones constructoras.

Ejemplo 7 – objetos enlazados

```
// Objeto normal que será utilizado como prototipo
let Usuario = {
    nombre: "",
    apellido: ""
}

// Función para inicializar los datos
Usuario.setDatos = function(nombre, apellidos) {
    this.nombre = nombre;
    this.apellidos = apellidos;
}

Usuario.saludar = function() {
    console.log(`Hola, soy ${this.nombre} ${this.apellidos}`);
}

// Se construyen objetos basados en el prototipo
let usuario1 = Object.create(Usuario);
let usuario2 = Object.create(Usuario);

// Se inicializan los objetos
usuario1.setDatos("David", "Pérez");
usuario2.setDatos("Jorge", "López");

// Función 'saludar'
usuario1.saludar(); // Hola, soy David Pérez
usuario2.saludar(); // Hola, soy Jorge López
```

3.8 Prototipos nativos

JS define un conjunto de prototipos **incluidos en el lenguaje**. Los más importantes son:

- **Object.prototype**. Es el prototipo base y todos los objetos están enlazados con él (a no ser que se construya un objeto sin prototipo). Define varios métodos, como **toString()**.

- **Function.prototype**. Es el prototipo con el que están enlazadas todas las funciones. Define entre otros, los métodos **call**, **apply** y **bind.**

3.8 Prototipos nativos

- **Prototipos de primitivas**. Como por ejemplo **String** y **Number.**

- **Array.prototype**. Prototipo con el que se enlazan los arrays y define sus métodos y propiedades (**length**, **pop**, **push**, **splice**, **concat**, etc.).

- **Date.prototype**. Es el prototipo con el que se construyen los objetos de tipo **Date**. Define métodos y propiedades relacionados con los objetos de fecha, como **getDate**, **getMinutes**, **getHours**, etc.

3.8 Prototipos nativos

Puedes ver las **propiedades definidas en los prototipos** tanto en la consola de Node como en la del navegador. Para ello, escribe el nombre del prototipo (por ejemplo Date.prototype). En NodeJS se pueden ver con **Object.getOwnPropertyNames(Date.prototype)**.

```
> Date.prototype
< {getTimeAlias: ƒ, constructor: ƒ, toString: ƒ, toDateString: ƒ, toTimeString: ƒ, …}
    getTimeAlias: ƒ getTime()
    constructor: ƒ Date()
    getDate: ƒ getDate()
    getDay: ƒ getDay()
    getFullYear: ƒ getFullYear()
    getHours: ƒ getHours()
    getMilliseconds: ƒ getMilliseconds()
    getMinutes: ƒ getMinutes()
    getMonth: ƒ getMonth()
    getSeconds: ƒ getSeconds()
    getTime: ƒ getTime()
    getTimezoneOffset: ƒ getTimezoneOffset()
    getUTCDate: ƒ getUTCDate()
```

```
> Object.getOwnPropertyNames(Date.prototype)
[
  'constructor',        'toString',          'toDateString',
  'toTimeString',       'toISOString',       'toUTCString',
  'toGMTString',        'getDate',           'setDate',
  'getDay',             'getFullYear',       'setFullYear',
  'getHours',           'setHours',          'getMilliseconds',
  'setMilliseconds',    'getMinutes',        'setMinutes',
  'getMonth',           'setMonth',          'getSeconds',
  'setSeconds',         'getTime',           'setTime',
  'getTimezoneOffset',  'getUTCDate',        'setUTCDate',
  'getUTCDay',          'getUTCFullYear',    'setUTCFullYear',
  'getUTCHours',        'setUTCHours',       'getUTCMilliseconds',
  'setUTCMilliseconds', 'getUTCMinutes',     'setUTCMinutes',
  'getUTCMonth',        'setUTCMonth',       'getUTCSeconds',
  'setUTCSeconds',      'valueOf',           'getYear',
  'setYear',            'toJSON',            'toLocaleString',
  'toLocaleDateString', 'toLocaleTimeString'
]
>
```

3.9 Clases

- JS ha incorporado una **nueva sintaxis** para trabajar con objetos. Dicha sintaxis facilita la tarea a personas que vengan de Java u otro lenguaje orientado a objetos que use clases.

- En JS no existen clases en el sentido clásico de otros lenguajes orientados a objetos. **Sólo existen objetos relacionados mediante prototipos**.

Ejemplo 8 – clases

Ejemplo 9 – clases extends super

```
class Usuario {
    constructor(nombre, apellidos) {
    this.nombre = nombre;
    this.apellidos = apellidos;
    }
    // No hay coma entre las definiciones de los métodos

    // 'saludar' está definido en 'Usuario.prototype'
    saludar() {
    console.log(`Me llamo ${this.nombre} ${this.apellidos}`);
    }
}

// Se crea un objeto, 'UsuarioEspecial.prototype', cuyo prototipo es 'Usuario'
class UsuarioEspecial extends Usuario {

    // 'cantar' está definido en 'UsuarioEspecial.prototype'
    cantar() {
    console.log("Yo también sé cantar");
    }
}

// Se crea un objeto cuyo prototipo es 'UsuarioEspecial'
let usuario = new UsuarioEspecial("David", "Pérez");

// JavaScript busca 'saludar' en el objeto ('usuario') y no lo encuentra
// A continuación busca en su prototipo, 'UsuarioEspecial' y tampoco lo encuentra
// Por último busca en el prototipo de 'UsuarioEspecial', que es 'Usuario' y finalmente lo encuentra
// 'saludar' está definido en 'Usuario.prototype'
// La ruta de búsqueda es: usuario -> UsuarioEspecial -> Usuario
usuario.saludar(); // Me llamo David Pérez

// JavaScript busca 'saludar' en el objeto ('usuario') y no lo encuentra
// A continuación busca en su prototipo, 'UsuarioEspecial', donde lo encuentra
// 'cantar' está definido en 'UsuarioEspecial.prototype'
// La ruta de búsqueda es: usuario -> UsuarioEspecial
usuario.cantar(); // También sé cantar
```

3.10 JSON

JSON (*JavaScript Object Notation*), es un formato **basado en texto** para representar objetos. Dada su importancia para enviar y recibir datos entre clientes y servidores, JS define los métodos:

- **JSON.stringify**: Convierte objetos a JSON
- **JSON.parse**: Convierte JSON a objetos JS

```
let usuario = {
    nombre: "Miguel",
    edad: 25,
    permisos: ["admin", "editor"]
}

let cadenaJson = JSON. stringify(usuario);
// cadenaJson es un string

let usuarioRecuperado = JSON.parse(cadenaJson);
// usuarioRecuperado es un objeto
```

4. Módulos

4.1 Tipos de módulos

- Tradicionalmente la carga de programas mediante ficheros externos se ha hecho con las etiquetas **<script>**.

- Esos ficheros definían un conjunto de variables y funciones que se incorporaban **directamente al objeto global**, con sus inconvenientes:

  - Conflictos entre nombres de funciones o variables definidas con el mismo nombre en distintos archivos.

  - Modificación voluntaria o involuntaria de variables y funciones definidas en otros ficheros.

4.1 Tipos de módulos

- Por ello surge el **concepto de módulo**. Un módulo es un fichero JS que permite definir **qué objetos (en forma de variables o funciones) se exportarán** para poder ser usados en el objeto global u otros módulos.

- Existen distintos sistemas de módulos:
  - **CommonJS**: El que se definió para NodeJS
  - **AMD**: Usado en navegadores por librerías de carga de módulos
  - **UMD**: Sistema universal compatible con CommonJS y AMD
  - **ES6**: Sistema de módulos incorporados al estándar **EcmaScript**. Compatible con navegadores actuales. En NodeJS se puede usar si los archivos usan la extensión .mjs en vez de .js

- Nos vamos a centrar en el sistema de módulos **ES6**, el más reciente.

4.2 import y export

- Este sistema define dos directivas:
  - **import** se usa para importar funcionalidad desde otros módulos. Sólo se podrán importar objetos de un módulo si han sido exportados por éste.
  - **export** se usa para definir qué objetos de los definidos en el módulo **serán accesibles desde el exterior**, es decir, podrás ser importados por el código que utilice dicho módulo.
- Ejemplo de código de un módulo guardado en el archivo **miLibreria.js**

```
1  // miLibreria.js
2  function saludar(usuario) {
3      console.log(`Hola, ${usuario}`);
4      return `Hola, ${usuario}`
5  }
6
7  function secreto()  {
8      console.log("no se puede decir, es un secreto");
9      return "no se puede decir, es un secreto"
10 }
11
12 function despedir(usuario)  {
13     secreto();
14     console.log(`Adiós, ${usuario}`);
15     return `Adiós, ${usuario}`
16 }
17
18 // Exportación de objetos
19
20 export {
21     saludar,
22     despedir
23 }
```

4.2 import y export

- Ejemplo de código de un programa **principal.js** que utiliza el módulo definido anteriormente:

```
1  // Fichero principal.js
2
3  // Importación de las funciones exportadas por el módulo
4  // También se puede indicar las funciones que se deseen importar,
5  // que no tienen porqué ser todas
6
7  import {saludar, despedir} from './miLibreria.js'
8
9  saludar("Juan"); // Hola, Juan
10 despedir("Juan"); // Adiós, Juan
```

Importación de toda la funcionalidad de un módulo:

```
1  // Fichero principal.js
2
3  import * as miLibreria from './miLibreria.js'
4
5  // El objeto 'miLibreria' tiene definidas todas las funciones
6  // exportadas por la libreria
7  miLibreria.miLibreriasaludar("Juan"); // Hola, Juan
8  miLibreria.despedir("Juan"); // Adiós, Juan
```

4.3 Módulos en HTML

- Para poder usar modulos en HTML y poder usar **import** para cargarlos es necesario usar el atributo **type="module"** en la etiqueta **<script>**

```
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>Document</title>
</head>
<body>
<script  type="module">
    import * as miLibreria from './miLibreria.js';

    alert(miLibreria.saludar("Juan"));
    alert(miLibreria.despedir("Juan"));

</script>
</body>
</html>
```

4.4 Librerías externas: CDN

- Gracias al sistema de módulos, podemos acceder a un gran número de **librerías externas** creadas por otras personas. JS tiene una comunidad de desarrolladores enorme, posibilitando una gran oferta.

- La mayoría de las librerías más usadas ofrecen compatibilidad con **import**, aunque no todas: es posible que algunas de ellas nos pidan que las utilicemos cargándolas en el objeto global mediante una etiqueta **<script>** convencional.

4.4 Librerías externas: CDN

- Para cargar las librerías externas existen dos posibilidades:
  - Descarga del fichero/s para almacenarlos junto con nuestro código.
  - Acceso al fichero/s a través de una **URL** sin necesidad de descargar. En este caso la URL suele apuntar a un **CDN**.

Un **CDN**, o **Content Delivery Network** es un **servicio distribuido de alojamiento de ficheros**. Su principal función es **garantizar la descarga de dichos ficheros de manera óptima**.

5.Programación funcional

5.1 Fundamentos: la función filter

- Es habitual trabajar con arrays para modificarlos, por ejemplo:
  - **Realizar un filtrado** para obtener un subconjunto de elementos que cumplan una condición
  - **Buscar** un determinado elemento
  - **Modificar** un array para transformarlo en otro con otras propiedades derivadas de las primeras
  - **Realizar un cálculo** con los elementos del array
  - **Realizar agrupaciones** de elementos del array, en función de alguna condición

5.1 Fundamentos: la función filter

- **Típicamente** estas operaciones se realizan de **modo iterativo** con bucles que recorran todos los elementos del array →

```
let datos = [14, 22, 16, 3, 35];

// Obtener los elementos menores que 15

let resultado = [];
for (let dato of datos) {
    if (dato < 15) {
        resultado.push(dato);
    }
}

console.log(resultado); // [14, 3]
```

En este ejemplo, se modifica la variable resultado de forma indirecta.

5.1 Fundamentos: la función filter

- A continuación vamos a ver la **alternativa funcional** al ejemplo anterior:

```
let datos = [14, 22, 16, 3, 35];

// Obtener los elementos menores que 15

let resultado = datos.filter(function(dato){
    return dato < 15;
}

console.log(resultado); // [14, 3]
```

- En la versión funcional hay una **clara asignación a la variable resultado**, esto facilita su comprensión. Veamos una tercera versión:

```
let datos = [14, 22, 16, 3, 35];

// Obtener los elementos menores que 15

let menorQue15 = dato => dato < 15 // función flecha

// Devuelve únicamente true o false y se pasa como parámetro a otra función
// Este tipo de funciones se llaman funciones predicado, y las funciones
// flecha son muy adecuadas para crearlas.

let resultado = datos.filter(menorQue15);

console.log(resultado); // [14, 3]
```

- La función que se pasa como parámetro a **filter** debe aceptar **3 parámetros**:
- 1. El **elemento que se está procesando** del array (**obligatorio**), 2. **la posición** y 3. **el array original**

```
let results = arr.filter(function(item, index, array) {
  // si devuelve true, el elemento es ingresado al array y la iteración continua
  // si nada es encontrado, devuelve un array vacío
});
```

5.2 find

- Devuelve el primer elemento que cumpla la condición pasada en la función:

```
let usuarios = [
    {id: 1, nombre: "Guillermo", apellido: "Martínez", nif: "11234432E"},
    {id: 2, nombre: "Juan", apellido: "Moreno", nif: "33422376F"},
    {id: 3, nombre: "Pablo", apellido: "Rodríguez", nif: "84732567J"}
]

let usuarioConId2 = usuarios.find(item => item.id == 2);
console.log(usuarioConId2);
// {id: 2, nombre: "Juan", apellido: "Moreno", nif: "33422376F"}
```

- La función que se pasa como parámetro a **find** es de nuevo una **función predicado**.

- La función que se pasa como parámetro a **find** debe aceptar **3 parámetros**:
- 1. El elemento que se está procesando del array (**obligatorio**), 2. **índice** y 3. el **array original**

```
let result = arr.find(function(item, index, array) {
  // si true es devuelto aquí, find devuelve el ítem y la iteración se detiene
  // para el caso en que sea false, devuelve undefined
});
```

5.3 forEach

- Permite ejecutar una función en cada uno de los elementos del array.

```
let usuarios = [
    {id: 1, nombre: "Guillermo", apellido: "Martínez", nif: "11234432E"},
    {id: 2, nombre: "Juan", apellido: "Moreno", nif: "33422376F"},
    {id: 3, nombre: "Pablo", apellido: "Rodríguez", nif: "84732567J"}
]


let nombres = ["Guillermo", "Juan", "Pablo"];

// se muestra cada nombre con alert
nombres.forEach(alert);

// se muestra con alert nombre y apellido de cada usuario
usuarios.forEach(item => alert(item.nombre + " " + item.apellido));

// se imprime cada item del array (objetos)
usuarios.forEach(function(item, indice, array){
        console.log(item);
});
```

- La **función que se pasa como parámetro** a ForEach **no es una función predicado, porque no devuelve true/false**

- La función que se pasa como parámetro a **forEach** debe aceptar **3 parámetros**:
- 1. El **elemento que se está procesando del array (obligatorio)**, 2. **índice** y 3. el **array original**

```
arr.forEach(function(item, index, array) {
  // ... hacer algo con el elemento
});
```

5.4 map

- Permite transformar arrays.

```
let numeros = [1, 3, 5, 7];

let dobleNumeros = numeros.map(function(numero){
    return numero*2;
});

// No se modifica el array original
console.log(numeros);
console.log(dobleNumeros);
```

- Cuidado al trabajar con arrays de objetos, pues la **función que se pasa a map puede realizar modificaciones sobre dichos objetos**. Si esto ocurre, **es posible que se devuelva un nuevo array**, **cuyos elementos apunten a los mismos objetos originales**. Si es lo que queremos no pasa nada, pero **si lo que queremos es que map devuelva un array de objetos distintos, entonces habría que hacerlo de otra manera**.

- La función que se pasa como parámetro a **map** debe aceptar **3 parámetros**:
- 1. El **elemento que se está procesando del array (obligatorio)**, 2. **índice** y 3. **el array original**

```
let result = arr.map(function(item, index, array) {
  // devuelve el nuevo valor en lugar de item
});
```

Ejemplo 1 – map

Ejemplo 2 – map

5.5 reduce

- **Recorre todos los elementos del array para generar un único valor** (**puede ser un objeto o un array también**).
- Uso: **realizar cálculos** (totales, medias, etc.) **o agrupaciones de datos**.

Parámetro 1	Parámetro 2
función callback	**un valor inicial** para la **primera ejecución del callback** (la primera vez que ejecuta **callback** no hay resultado de la ejecución anterior). Parámetro opcional. Habitualmente es un valor vacío: **“ ”**, **0**, **[ ]** o **{ }**

- Es la **función más potente** de todas las vistas. **Todas las demás se pueden implementar a partir de ésta.**

```
let arrNumeros = [2, 5, 4];
let total = arrNumeros.reduce(function(acc,numero){
                        return acc +  numero;
                                }, 0);

console.log(total);
```

```
let value = arr.reduce(function(accumulator, item, index, array) {
  // ...
}, [initial]);
```

La **función callback**, admite **4 parámetros**:

- El **resultado de la ejecución anterior del callback (acumulador) [su valor inicial es el segundo parámetro de reduce]**
- **Elemento en proceso**
- **Índice**
- **Array original**

Ejemplo 3 – reduce

6. JavaScript moderno

6.1 Modo estricto

- Característica aparecida en el estándar **EcmaScript5, de 2011** con la intención de **convertir en errores explícitos**, **situaciones que generaban errores silenciosos o efectos no deseados** al ejecutar los programas. Se puede activar:
  - **A nivel de script**, indicándolo al principio del archivo
  - **A nivel de función**, indicándolo en la primera línea del cuerpo de la función
  - En los **archivos de módulo**, dónde está activado **por defecto** (no hay que hacer nada para activarlo en módulos)

- **No es buena idea mezclar código en modo normal y modo estricto**. Por ello, es más recomendable usar el modo estricto **a nivel de script**.

- Se activa con **"use strict";** o **'use strict';** en la **primera línea** del archivo de script o función.

6.2 ¿Clases u objetos?

- Aunque JS permite diferentes maneras de trabajar con objetos (**funciones constructoras** [uso de **new**], **objetos enlazados** [**Object.create**], y por último **clases**, es recomendable mantener la coherencia y usar la misma técnica en todo el código de la aplicación.

- Conviene saber que **algunas API o librerías** nos fuerzan a usar un método concreto (como por ejemplo **Web Components** [un estándar en desarrollo para crear componentes HTML que puedan ser reutilizados en las páginas] que obliga a usar la **sintaxis de clases**.

6.3 Manejo de errores

- Se realiza con los bloques **try…catch**. Funcionan de modo parecido a otros lenguajes. Los errores que se capturan son los que aparecen en tiempo de ejecución.
- La función catch define un parámetro, que es el error capturado. Dicho parámetro hace referencia a un objeto **Error**, que tiene tres propiedades:
  - **name**. El nombre del error
  - **message**. El mensaje de error
  - **stack**. Información sobre la ruta de ejecución que ha provocado el error

```
1  try{
2      // Acceso a una propiedad de una variable que no existe
3      console.log(hola.noexiste);
4  } catch(error){ // El parámetro puede tener cualquier nombre, no tiene porqué ser 'erro
5  r'   console.log(error.name); // ReferenceError
6      console.log(error.message); // hola is not defined
7  }
```

6.3 Manejo de errores

- También es posible generar **errores personalizados** mediante **throw**:

```
try{
    let a = "valornovalido";

    if (a == "valornovalido"){
        // Generamos un error con un mensaje personalizado
        throw new Error("a no puede tener ese valor");
    }
} catch(err){
    console.log(err.name); // Error
    console.log(err.message) // a no puede tener ese valor
}
```

- **try…catch** funciona de manera síncrona. Por tanto, no funciona correctamente con **funciones asíncronas**.

6.3 Manejo de errores

- A veces puede ser interesante **ejecutar un trozo de código independientemente de que se haya generado un error o no**.

- En el siguiente ejemplo se hace una **petición a un recurso remoto cuya conexión hay que cerrar**, tanto si se realiza con éxito como si no. En ese caso podemos usar el bloque **finally**:

```
try{
    // Apertura de conexión
    // Procesamiento de datos
} catch(err) {
    // Captura de errores
} finally {
    // Cierre de conexión
    // Este bloque se ejecuta siempre, tanto si hay error como si no
}
```

6.3 Manejo de errores

- A veces puede ser interesante ejecutar un trozo de código **independientemente de que se haya generado un error o no**. En el siguiente ejemplo se hace una petición a un recurso remoto cuya conexión hay que cerrar, tanto si se realiza con éxito como si no. En ese caso podemos usar el bloque **finally**:

```
try{
    // Apertura de conexión
    // Procesamiento de datos
} catch(err) {
    // Captura de errores
} finally {
    // Cierre de conexión
    // Este bloque se ejecuta siempre, tanto si hay error como si no
}
```

6.4 Evolución y futuro

- JS está en continua evolución. Es muy popular y además está estrechamente ligado a la web. Algunas de las **características** que se han añadido en los **últimos años** son:

Operador rest y sintaxis spread	**Promesas y funciones asíncronas**
Generadores (no lo estudiaremos)	Proxies (no lo estudiaremos)
El sistema de módulos	Tipo de datos BigInt
Nuevas estructuras y sintaxis	Mejoras de sintaxis para la simplificación de estructuras de código

- También hay una **evolución muy grande** de las **API de los navegadores**, por lo que se espera que evolucione en los próximos años incorporando nuevas características para crear aplicaciones cada vez más complejas y potentes.